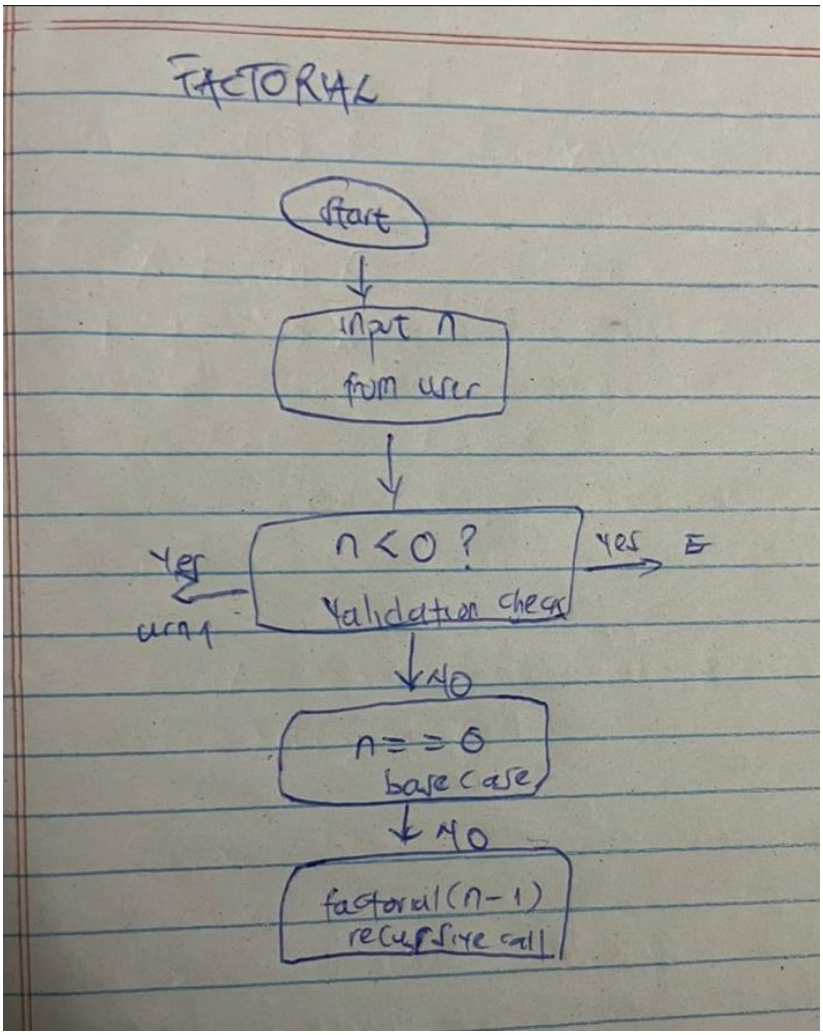
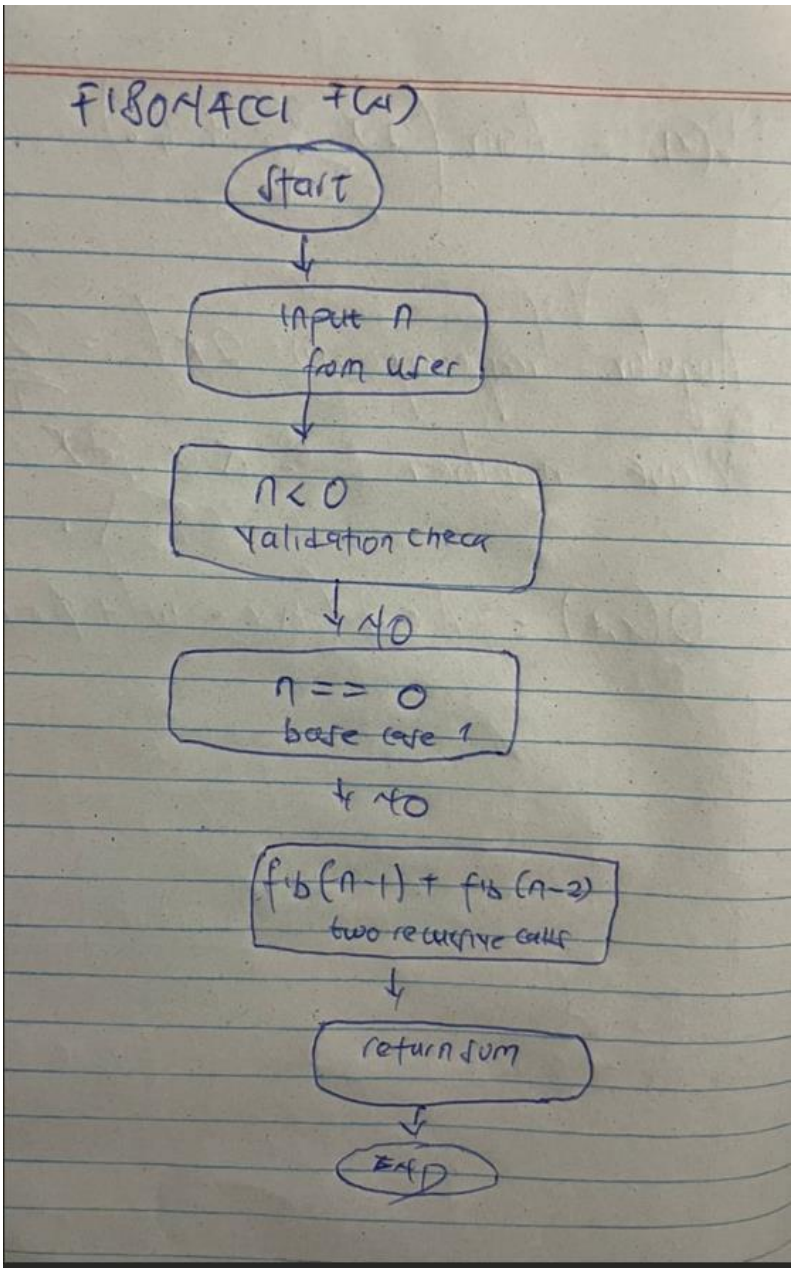
Question 1 flow chart





Pseudo Code:

(i) Factorial

```
    FUNCTION factorial(n):
  // Base case
  IF n == 0 THEN
    RETURN 1
  END IF

  // Recursive case
  RETURN n × factorial(n - 1)

END FUNCTION

// Main program
BEGIN
  PRINT "Enter a non-negative integer:"
  READ n

  IF n < 0 THEN
    PRINT "Error: n must be ≥ 0"
  ELSE
  result ← factorial(n)
    PRINT n, "! =", result
  END IF
END
```

(ii) Fibonacci

```
FUNCTION fibonacci(n):
  // Base cases
  IF n == 0 THEN
    RETURN 0
  END IF
  IF n == 1 THEN
    RETURN 1
  END IF

  // Recursive case
  RETURN fibonacci(n-1)
       + fibonacci(n-2)

END FUNCTION

// Main program
BEGIN
  PRINT "Enter a non-negative integer:"

  READ n

  IF n < 0 THEN
    PRINT "Error: n must be ≥ 0"
  ELSE
    result ← fibonacci(n)
    PRINT "F(", n, ") =", result
  END IF
END
```

Python code :

```
# —— ICT102 Activity – Recursion: Factorial & Fibonacci ——
# Student: [Your Name / Student Number]
# Date   : April 2026

def factorial(n):
    """Return n! using recursion. n must be a non-negative integer."""
```

```
    if n == 0:          # base case
        return 1
    return n * factorial(n - 1)   # recursive case

def fibonacci(n):
    """Return the nth Fibonacci number using recursion."""
    if n == 0:          # base case 1
        return 0
    if n == 1:          # base case 2
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)  # recursive

def get_positive_int(prompt):
    """Prompt the user until a valid non-negative integer is entered."""
    while True:
        raw = input(prompt).strip()
        if not raw.lstrip('-').isdigit():
            print(f"  ✗ '{raw}' is not an integer. Please try again.\n")
            continue
        value = int(raw)
        if value < 0:
            print(f"  ✗ Negative numbers are not allowed. Please try again.\n")
            continue
        return value


def main():
    print("=" * 45)
    print(" ICT102 – Recursive Factorial & Fibonacci")
    print("=" * 45)

    # ── Factorial ──────────────────────────────
    print("\n── Factorial ──")
    n = get_positive_int("Enter n to compute n!: ")
    result = factorial(n)
    print(f"  {n}! = {result}\n")
```

```
    # — Fibonacci ————————————————————————————
    print("— Fibonacci —")
    n = get_positive_int("Enter n to compute F(n): ")
    result = fibonacci(n)
    print(f"  F({n}) = {result}")

    # Optionally print the full sequence up to n

    seq = [str(fibonacci(i)) for i in range(n + 1)]
    print(f"  Sequence: {', '.join(seq)}\n")

    print("=" * 45)

if __name__ == "__main__":
    main()
```